

***** Kraveo Deep Audit Report ***** Implementation, security, platform, product and launch-readiness audit for the VIT Bhopal food-delivery platform Prepared 6 August 2026 A. Read-only audit A. No source files changed Executive verdict. Kraveo is a compelling and unusually thoughtful campus-delivery prototype, but it is not safe or functional as a live food-delivery marketplace. Do not accept real money, collect real GPS, or onboard external users until the critical launch blockers in this report are resolved and independently tested. ***** 1. Executive scorecard *****

Dimension	Score	Assessment
Excellent VIT Bhopal-specific insight: gate handoffs, dhaba workflows and late-night runner	9.0 / 10	Product concept and campus fit
constraints are real differentiators. Prototype and interaction design	8.2 / 10	Strong persona-driven UI across customer, vendor, driver and admin surfaces.
Compiles and presents well, but critical Engineering MVP are local simulations rather than a connected product.	5.5 / 10	workflows
Unsafe identity, data access, payment, Backend production readiness	2.5 / 10	persist- tence and WebSocket behavior block real launch.
Requires the commerce, security, mobile Safe public-pilot readiness	1.5 / 10	integra- tion and operating foundations outlined below.

***** 2. Audit method and verified checks ***** Three specialist audits covered backend/security, Flutter/mobile platform, and web/ product/operations. The following checks were verified:

- * Backend TypeScript build passes; admin web production build passes.
- * Existing Flutter tests pass: customer 14, vendor 4, driver 3.
- * Current production Node dependency audits show no known vulnerabilities.
- * Flutter analysis reports 49 informational issues total: customer 28, vendor 17, driver 4.
- * There are no backend API tests, web tests, end-to-end tests, CI workflows, deployment infrastructure or Prisma migrations.
- * The running API uses in-memory stores; Prisma schema exists but Prisma is not instantiated or used.

***** 3. Capability truth matrix *****

Area	Current state	Launch implication
Polished local catalog/cart Customer app	simulation; checkout is blocked.	Cannot place a real order.
Vendor app	Strong KDS/stock UI with seeded local orders.	No real incoming order, alert or acknowledgement path.
Driver app	Scripted delivery demo	
Admin dashboard	Attractive mock command centre;	
delivery proof.		
many local-only mutations.	succeed when they have not.	
Express/Socket.IO prototype	Data loss on restart and no Backend and database	with in-mem- ory state; Prisma safe multi-user operation.
schema only.	Payments and notifications Razorpay/FCM-shaped code with	Not suitable for com- merce or
simulation fallbacks.	reliable alerting.	

***** 4. Critical launch blockers ***** Severity

Finding and evidence	Required directive	Severity
Privilege escalation. backend/src/ routes/api.ts accepts a role from caller only. Provision vendor, driver and admin can receive an ADMIN JWT.	Use verified phone-OTP authentication. Public registration creates students roles through audited back-office workflows.	Critical unauthenticated login input; a
Orders and PII are exposed. Order list/ ownership policy on every query. Return authentication and return customer, hostel, driver and drop-off data.	Require authentication and central role-specific, privacy-minimised DTOs.	Critical detail routes do not require
Any authenticated role can update any actor-bound commands: vendor accept/ actor or resource check.	Replace generic status updates with Critical order. Generic status mu- tation has no actor-bound commands: vendor accept/ prepare, driver pickup/arrive, server OTP delivery confirmation.	
Payment integrity is broken. Orders are marked PAID before verification; trusted; verified provider webhooks, idempotency, live gateway errors become simulated success.	Use payment-pending orders, integer pause, server-calculated amount, Critical arbitrary amount/order input is refunds and daily reconciliation.	
Customer checkout cannot finish. It state and test an end-to-end successful subsequent completion code is	Replace with cancellable payment/loading Critical awaits a non-dismis- sible dialog and checkout path.	

unreachable.

Mobile apps are disconnected local repository/state layer, sockets, HTTP, socket, auth or repository integration is present. retries, error states and durable local cache.

Delivery OTP is bypassable. Driver code Generate, hash, expire and verify OTP Critical accepts universal fallback codes and server-side for the assigned driver/customer OTP is locally generated. order with rate-limit and attempt logout.

Socket.IO is unauthenticated. Any origin Authenticate handshakes, derive rooms Critical can connect, join arbitrary rooms and server-side, remove client-authoritative emit global driver/order events. events and publish privacy-safe role DTOs only.

Release Android manifests have no Move required permission to main Critical INTERNET permission. Permission appears manifests and check merged release only in debug/profile manifests. manifests in CI.

All backend state is volatile. Runtime Implement PostgreSQL persistence, Critical uses mutable arrays/maps. transactions, optimistic concurrency and transactional outbox before a pilot. ***** 5. High-priority platform findings *****

**** Mobile and release ****

- * No push notification, app-killed alert, background job, map, GPS, navigation, payment SDK or offline synchronization implementation exists.
- * Vendor alarm depends on a remote audio URL; fallback only logs console text.
- * All Android apps use generic com.example.* IDs, generic branding and debug signing for release.
- * No iOS projects exist despite documentation presenting customer and driver iOS support.
- * Accessibility work is incomplete: no semantic labels, no TalkBack/font-scale test evidence and color-dependent status signals. ***** Admin and operations *****
- * Admin has no login, session expiry, MFA, role model, audit trail or staff approval workflow.
- * Status actions update UI optimistically but ignore rejected backend responses; vendor onboarding, toggle and driver reassignment are largely local-only.
- * The live map is visual only: driver latitude/longitude is not used for geographic positioning.
- * Analytics/KPIs are static data rather than defined operational metrics.
- * There is no observability stack: structured logs, traces, metrics, error reporting, alerting, backup restore test or incident runbook. ***** 6. Documentation and governance ***** Documentation strength: the architecture, UX and rollout documents are comprehensive. However, the README and several documents present planned/simulated capabilities as currently operational. Introduce an IMPLEMENTED / SIMULATED / PLANNED capability matrix immediately.
- * Remove a??production-readya?? wording until launch gates are satisfied.
- * Add an OpenAPI reference, architecture decision records, threat model, data inventory, retention policy, support/refund/cancellation runbooks and incident/disaster-recovery plan.
- * Before launch, obtain appropriate professional review for vendor food-safety obligations, university/gate permission, privacy/data protection, consumer terms/refunds, runner safety/insurance, tax/settlement and payment compliance. ***** 7. Recommended target architecture ***** Use a well-tested modular monolith first. Do not add microservices before the core order lifecycle is reliable.

Layer	Target responsibility
Typed authenticated client, encrypted session storage, durable Flutter apps	local cache/outbox,
role-specific UX and reliable error/reconnect states. API backend	Identity, catalog, order, dispatch, payment, notification, admin/support modules with a central authorization policy. PostgreSQL
memberships, vendors, menus, orders,	Source of truth for users,
payments, payouts, OTP attempts and immutable audit/order events. Redis and worker queue	Socket adapter, cache, presence, delayed tasks, notification
retries and dead-letter processinga??not the source of truth. Integrations	Verified Razorpay webhooks, Firebase Admin/FCM, media object
storage/CDN and maps/routing provider. Operations	OpenTelemetry, error reporting,
health/readiness, metrics, alerts,	
backups, reconciliation jobs and runbooks. Core rules: integer paise for money; UUID/ULID identifiers; transactional commands; idempotency keys; immutable event/audit history; transactional outbox; authenticated server-authorized sockets; OpenAPI contracts; privacy-minimised response DTOs. ***** 8. Required order lifecycle *****	

**** Cart a?? Quote calculated a?? Payment pending a?? Payment verified a?? Vendor notified a?? Vendor accepted/rejected/timed out a?? Preparing a?? Ready for pickup a?? Driver offered a?? Driver assigned a?? Picked up

a?? Arrived at gate a?? OTP verified a?? Delivered At valid stages, separately model cancellation, payment failure, refunds, support disputes, driver reassignment, vendor timeout and safety incidents. Every command must record actor, timestamp, reason, policy check, idempotency key and audit event. ***** 9. Execution roadmap *****
Phase 0 a?? Stop-the-line fixes: 1a??2 weeks *****

1. Remove tracked environment files from Git history and rotate every potentially exposed secret.
2. Fix checkout and release network permissions.
3. Delete universal OTP bypasses, demo progression controls, payment-success fallbacks and development-admin token fallback.
4. Publish a truthful capability matrix and revise production-ready claims.
5. Set up protected main branch, reviews, environment separation, secret handling and baseline CI. Exit gate: no privilege escalation, no fake paid order path, clean release manifest, truthful documentation and automated build/analyze/test checks. ***** Phase 1 a?? Trusted commerce core: 3a??6 weeks *****

1. Implement PostgreSQL, Prisma migrations, repositories, transactions and seed data validation.
2. Build phone OTP identity, server-managed roles and vendor operator memberships.
3. Make server authoritative for catalog, quote, price, coupons, fees, order, delivery OTP and payment records.
4. Implement verified Razorpay webhook lifecycle, refunds and reconciliation ledger.
5. Add authorization tests, API integration tests, payment tests, migration tests, rate limiting, security headers and structured logs. Exit gate: no cross-user access, all payment events reconcile, and restarts preserve all business data. ***** Phase 2 a?? Real fulfilment: 4a??8 weeks *****

1. Connect mobile apps to authenticated API and Socket.IO clients.
2. Build FCM vendor alerts with bundled alarm assets, retry and acknowledgement behavior.
3. Build driver availability, offer/accept/expiry, assignment locking, earnings ledger and delivery proof.
4. Add consented location, map/routing, geofences, freshness detection and privacy retention controls.
5. Convert admin into a role-based, server-authoritative, auditable operations console. Exit gate: a real order can travel from customer through verified payment, vendor, driver and OTP delivery, including recoverable connectivity failure. ***** Phase 3 a?? Controlled campus pilot: 3a??4 weeks *****

1. Begin with one small service zone, limited hours, small vendor set and manual-dispatch fallback.
2. Obtain campus/gate approval, vendor compliance evidence, packaging process, runner vetting and safety escalation plan.
3. Establish cancellation/refund, support, food complaint, outage and rider incident runbooks.
4. Use staging with test payments, push, maps, seed data, monitored dashboards and a verified backup restore drill. ***** Phase 4 a?? State-of-the-art optimisation *****

- * ETA and prep-time prediction; capacity-aware dispatch and batching.
- * Geofenced service zones, demand-aware delivery fees and vendor SLA management.
- * Runner utilization, safety-aware dispatch and fair earnings systems.
- * Personalization and experiments only after reliable fulfilment.
- * Operational analytics: fill rate, acceptance, prep SLA, dispatch lag, ETA error, delivery success, cancellation, refund, contribution margin and repeat rate. ***** 10. Final directive ***** Do not spend the next month adding more screens. Preserve the excellent UX already built and focus on: persistent data, secure identity/ownership, verified payments, real mobile connectivity, reliable dispatch/location/notifications, and tested pilot operations. That is the path from convincing demo to defensible food-delivery business. Kraveo Deep Audit Report A. Prepared from direct repository inspection and specialist read-only audits A. 6 August 2026